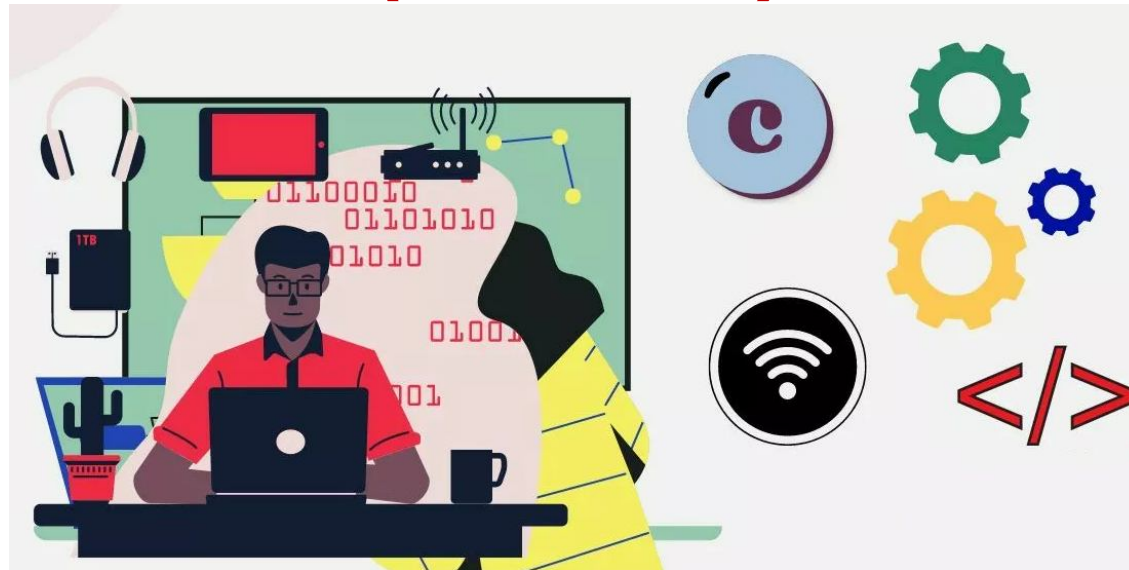


Structured Programming Concepts

(26CSE0105)



C Program Organization

Prepared By: Dr. Punit Soni

Department of Computer Science and Engineering

Chitkara University Institute of Engineering and Technology

Chitkara University, Punjab

Topics to be covered

- ***Modular Programming***
- ***Functions as Building Blocks***
- ***Header Files,***
- ***Pre-processor Directives (#include, #define)***
- ***Standard C Libraries (stdio.h, stdlib.h, string.h, math.h, ctype.h, time.h)***

Modular Programming Concept



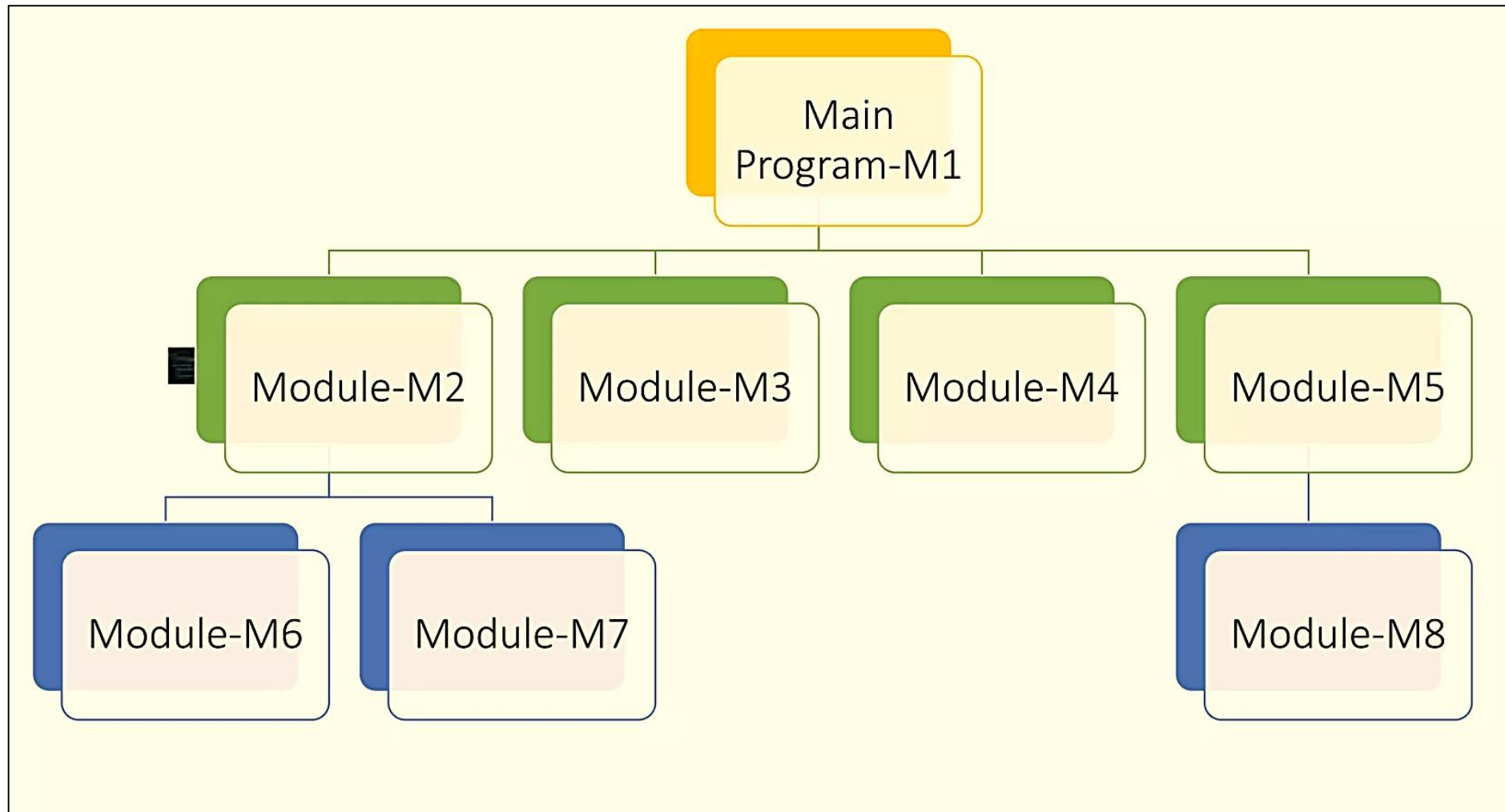
- **Modular programming** means splitting your code into **smaller, reusable parts/modules**.
- This makes your code **easier to read, maintain, and debug**.
- In **C**, these **modules** are commonly implemented using **functions**.

Advantages:

- **Simplicity**
- **Easy to Reuse**
- **Ease of Maintenance**
- **Team Collaboration**



Modular Programming Concept



Function as Building Blocks

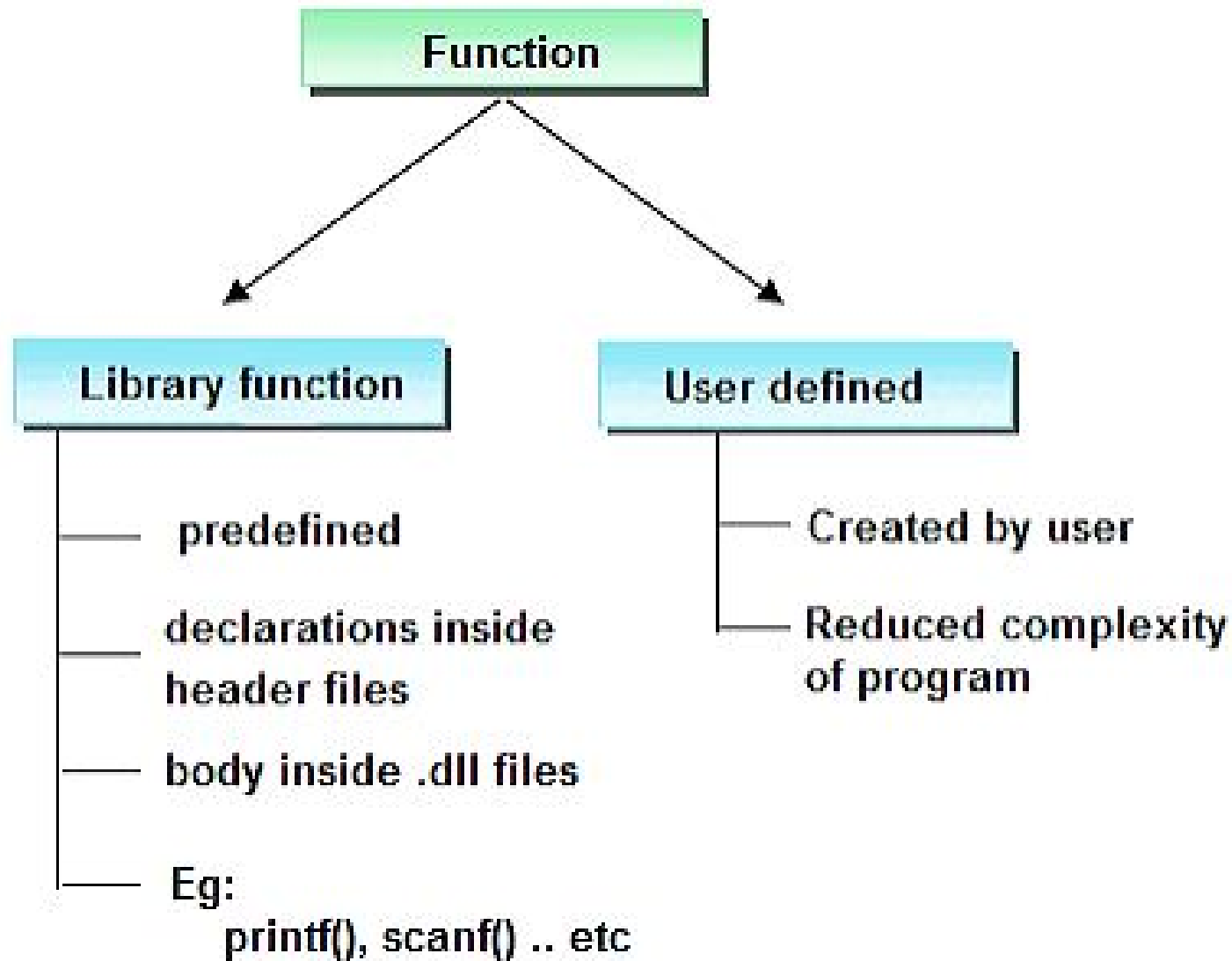
- A **function** is a **group of statements** that together perform a task.
- Every **C program** has **at least one function**, which is **main()**, and can have additional functions.
- A good programming principle is:

One function → **One specific responsibility**

Why Functions are "Building Blocks" ????

- **Modularity (Top-Down Design)**
- **Code Reusability**
- **Easier Debugging**

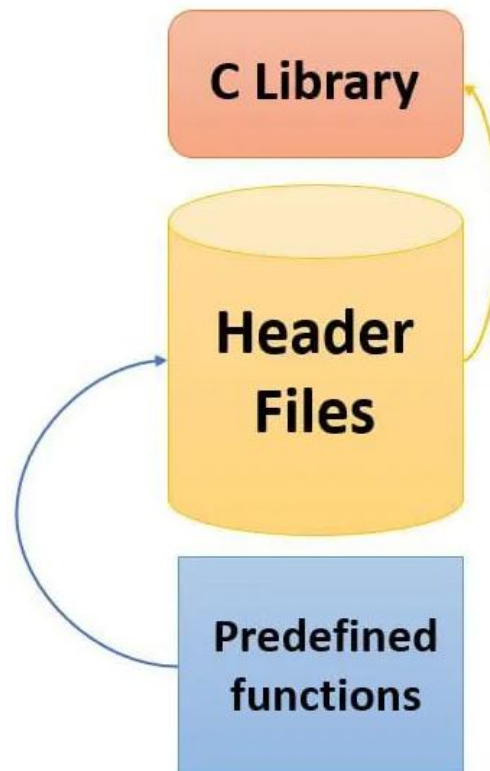
Function as Building Blocks



Header Files



- **Header files** in C are files with **‘.h’** extension that contain **function declarations**, and **macro definitions** that can be included in C programs using the **#include (preprocessor directive)**.



Various System Header Files

name	discription
<stdio.h>	Input/Output Functions
<conio.h>	console input/output
<math.h>	Mathematics Functions
<time.h>	Date and Time Functions
<string.h>	String Functions
<stdlib.h>	General Utility Functions
<complex.h>	A set of function for manipulating complex numbers
<ctype.h>	Character handling functions

Various System Header Files

- ***<stdio.h>***

function	discription
Printf()	This function is used to print the character, string, float, integer, octal and hexadecimal values onto the output screen
Scanf()	This function is used to read a character, string, numeric data from keyboard.

- ***<conio.h>***

function	discription
Clrscr()	This function is used to clear the output screen.
Getch()	It reads character from keyboard

Various System Header Files

- **<math.h>**

Function	Description
Pow()	This is used to find the power of the given number.
Sqrt()	This function is used to find square root of the argument passed to this function.
Log()	This function is used to calculate natural logarithm.
Exp()	This function is used to calculate the exponential "e" to the x th power.

- **<time.h>**

function	discription
Getdate()	This function is used to get the CPU time
Setdate()	This function used to modify the system date
Clock()	This function is used to get current system time
Time()	This function is used to get current system time as structure

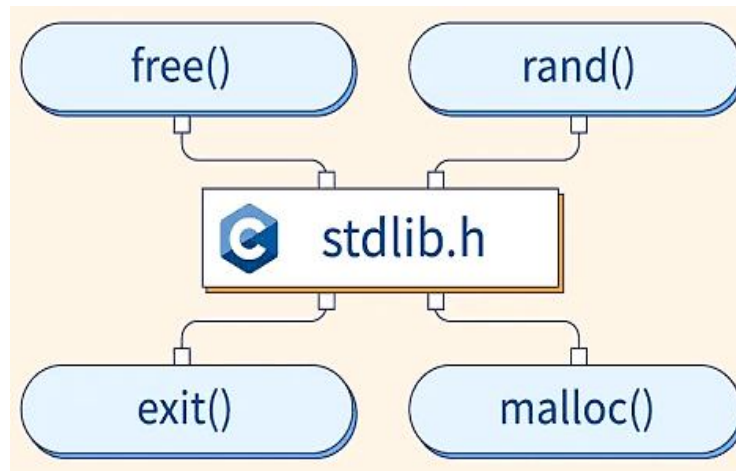
Various System Header Files

- *<string.h>*

function	discription
Strlen()	Calculates the length of string
Strcpy()	Copies a string to another string
Strcmp()	Concatenates(joins) two strings
Strcat()	Compares two string
Strlwr()	Converts string to lowercase
Strupr()	Converts string to uppercase

Various System Header Files

- ***<stdlib.h>***



- ***<ctype.h>***

Prototype	Function description
<code>int isdigit(int c);</code>	Returns a true value if <code>C</code> is a digit and 0 (false) otherwise.
<code>int isalpha(int c);</code>	Returns a true value if <code>C</code> is a letter and 0 otherwise.
<code>int isalnum(int c);</code>	Returns a true value if <code>C</code> is a digit or a letter and 0 otherwise.
<code>int islower(int c);</code>	Returns a true value if <code>C</code> is a lowercase letter and 0 otherwise.
<code>int isupper(int c);</code>	Returns a true value if <code>C</code> is an uppercase letter and 0 otherwise.
<code>int tolower(int c);</code>	If <code>c</code> is an uppercase letter, <code>tolower</code> returns <code>c</code> as a lowercase letter. Otherwise, <code>tolower</code> returns the argument unchanged.

Pre-processor Directives in C

- ***Preprocessor directives** are **commands** that instruct **C processor** to **modify source code** before the **actual compilation begins**.*
- *They always start with **hash symbol** (#).*

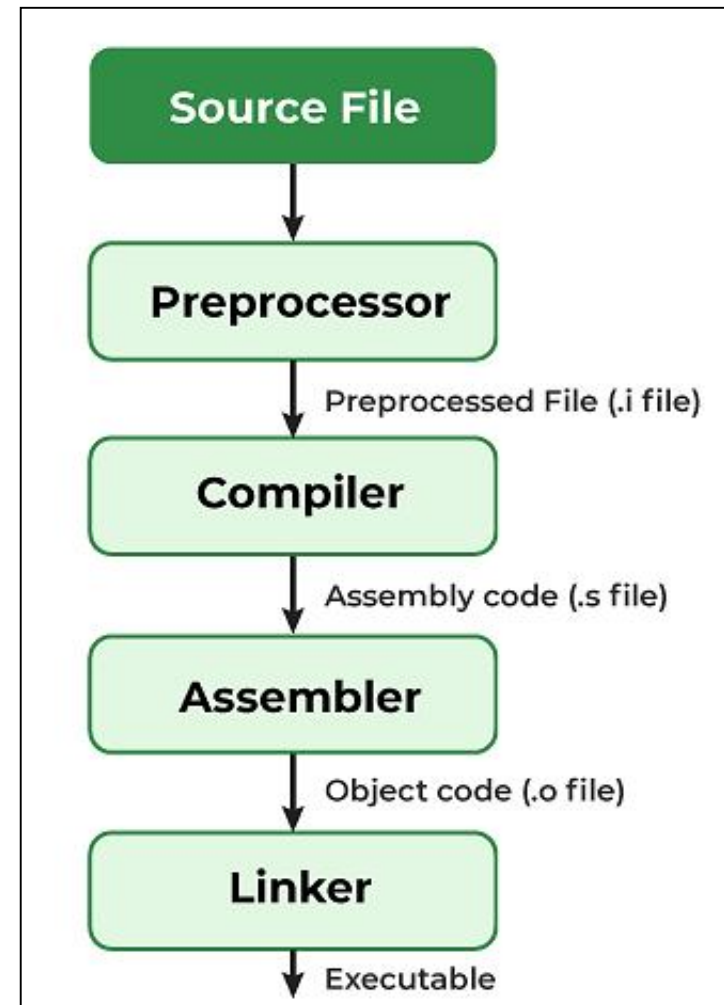
They are used for:

- ❖ ***Including header files***
- ❖ ***Defining constants and macros***
- ❖ ***Conditional compilation***
- ❖ ***File control***

How they work ???



- **Preprocessors** are **not part** of the **compilation process** but **operate separately**, allowing programmers to modify the code before compilation.
- Acts as a **text substitution tool**.
- This process happens entirely before the **source code** is **converted** into **object code**.



Types of Pre-processors



Macros

- **Macros** are used to define **constants** or **create functions** that the **preprocessor replaces throughout the code before the compilation**.
 - ❖ **#define** are used to **create** constant/macros.
 - ❖ **#undef** are used to **remove** constant/macros.

```
#include<stdio.h>
#define a 10
int main()
{
    printf("a = %d", a);
    return 0;
}
```

```
#include<stdio.h>
#define SQR(x) x*x
int main()
{
    int n;
    scanf("%d",&n);
    printf("%d", SQR(n));
    return 0;
}
```

File Inclusion

- **File inclusion** allows you to **include external files** (**header files, libraries, etc.**) into the program.

❖ Using **#include** directive, we can include files.

```
#include<stdio.h>
int main()
{
    printf("Hello World");
    return 0;
}
```

Searches in standard
system directories.

```
#include"stdio.h"
int main()
{
    printf("Hello World");
    return 0;
}
```

Searches in the current or
user-specified directory.

Conditional Compilation

- **Conditional compilation** allows you to **include** or **exclude** specific sections of code depending on **certain conditions**.
- This is useful for creating **platform-specific code** or for **debugging**.

There are the following conditional preprocessor directives:

- ❖ **#if, #else, #elif, #endif** - basic logic for **conditional inclusion**.
- ❖ **#ifdef** - Includes code if a **macro is defined**.
- ❖ **#ifndef** - Includes code if a **macro is not defined**.

Conditional Compilation

used for conditional compilation.

used with `#if` for checking multiple conditions.

Used with `#if` to provide an alternative block.

Marks the end `#if` block.

```
#include <stdio.h>
#define VALUE 2
int main()
{
    #if VALUE > 3
        printf("Value is greater than 3\n");
    #elif VALUE == 3
        printf("Value is 3");
    #else
        printf("Value is less than or equal to 2");
    #endif
    return 0;
}
```

Difference Between Macro and Function:

Macro	Function
Processed by preprocessor.	Processed by compiler.
Faster	Slightly slower
No type checking	Type checking available
No memory allocation	Uses memory during execution

Practice Questions



1. Header File – Mathematical Calculations

Write a C program that takes a number as input from the user and calculates its **square root, power, and absolute value**. Use the appropriate standard header file and its functions to perform these calculations. Identify which header file is required and include it using the `#include` preprocessor directive.

Sample Input:

Enter a number: 25

Enter power: 2

Sample Output:

Square Root: 5.00

Power: 625.00

Absolute Value: 25

Practice Questions



2. Preprocessor Directive – Symbolic Constant

A company wants to calculate the final price of a product after adding a fixed GST rate. The GST rate should be defined as a **symbolic constant using the #define preprocessor directive** instead of declaring it as a normal variable. Write a C program that takes the product price as input and calculates and displays the GST amount and final price.

Sample Input:

Enter product price: 1000

Sample Output:

GST Amount: 180.00

Final Price: 1180.00

Concept: #define, symbolic constants, arithmetic operators.



Thank You!

